Assignment 4: Maximum Concurrent Flow and Network Design

CS 536 – Data Communication and Computer Networks, Spring 2026

Pranay Nandkeolyar and Arav Popli

April 24, 2026

Preliminaries

We have a directed network G with n nodes and unit-capacity links, where every node has exactly d outgoing and d incoming links. Given a traffic matrix T (with $T_{ii} = 0$), the *maximum concurrent flow* $\alpha^*(G, T)$ is the largest factor α by which we can scale T and still route it without exceeding any link's capacity. Intuitively, $\alpha^* = 0.7$ means we can deliver 70% of the requested traffic simultaneously. The goal is to design G to make α^* as large as possible.

Two upper bounds on α^*. Both bounds come from a simple observation: the network has limited capacity, so there is a ceiling on how much traffic can flow.

Hop bound. Since each node only has d outgoing links, it can only connect directly to d neighbors; all other destinations require multi-hop paths. The more hops traffic takes, the more link capacity it consumes, which limits how much total demand can be routed. Summing link capacities gives:

$$\alpha^*(G, T) \;\leq\; \frac{nd}{\displaystyle\sum_{s \neq t} T_{s,t}\, d_G(s,t)}. \tag{1}$$

Cut bound. Any set of nodes S can only send traffic to the rest of the network through the links that cross the cut $(S, \bar{S})$:

$$\alpha^*(G, T) \;\leq\; \min_{S} \frac{\text{links crossing cut } (S, \bar{S})}{\displaystyle\sum_{s \in S,\, t \notin S} T_{s,t}}. \tag{2}$$

Part 1 – Uniform Traffic Matrix (50 pts)

The uniform traffic matrix is

$$T_{ij} = \frac{d}{n-1} \quad (i \neq j), \qquad T_{ii} = 0.$$

Each row and column sums to exactly d, fully using each node's link budget. This is the all-to-all pattern from AllGather/AllReduce steps where every node communicates with every other node at the same rate.

Step 1: An upper bound from max-flow min-cut

Think of the network as a shared pool of nd unit-capacity link-slots (each of the n nodes contributes d outgoing links). Routing demand pair (s,t) at rate αT_{st} consumes at least $\alpha T_{st} \cdot d_G(s,t)$ link-slots, because the flow must traverse at least $d_G(s,t)$ edges. Summing over all pairs and requiring that total consumption cannot exceed total supply gives:

$$\alpha \sum_{s \neq t} T_{st}\, d_G(s,t) \;\leq\; nd.$$

Rearranging and substituting $T_{st} = d/(n-1)$:

$$\alpha^* \;\leq\; \frac{nd}{\dfrac{d}{n-1} \cdot n(n-1) \cdot \bar{h}(G)} \;=\; \frac{1}{\bar{h}(G)},$$

where $\bar{h}(G) = \frac{1}{n(n-1)} \sum_{s \neq t} d_G(s,t)$ is the average hop count. **Maximizing α^* is therefore equivalent to minimizing the average hop count.**

This is a max-flow min-cut argument in disguise: the "cut" is the entire link budget of the network, and the bottleneck is how many times each unit of demand must cross links on its way from source to destination. Any topology that forces traffic to take long paths wastes link capacity and reduces α^*.

Step 2: A lower bound on average hop count (impossibility)

How small can $\bar{h}(G)$ be? Since $\bar{h} \leq D(G)$, it suffices to lower-bound the diameter. In any d-regular network, a BFS from a source discovers at most d nodes at depth 1 and at most $d(d-1)^{k-1}$ new nodes at depth $k \geq 2$ (each neighbor exposes at most $d-1$ previously unseen nodes). For all n nodes to be reachable within diameter D:

$$n \;\leq\; 1 + d \sum_{k=0}^{D-1} (d-1)^k \;\approx\; d \cdot (d-1)^{D-1},$$

which forces $D \geq \log_{d-1} n$ (approximately). Hence $\bar{h}(G) \geq \log_{d-1} n$ and:

$$\alpha^* \;\leq\; \frac{1}{\log_{d-1} n}. \tag{3}$$

No d-regular topology can beat this bound, no matter how cleverly it is wired.

Step 3: The De Bruijn digraph achieves the bound

Construction. The De Bruijn digraph $\mathrm{DB}(d,k)$ has $n = d^k$ nodes, each represented as a length-k string over $\{0, \ldots, d-1\}$. The edge rule is: drop the leftmost character, append any new character on the right. Formally, from node $b = (b_{k-1}, \ldots, b_0)$ there is a directed edge to each of the d nodes

$$b \;\longrightarrow\; (b_{k-2}, \ldots, b_0,\, c) \quad \text{for each } c \in \{0, \ldots, d-1\}.$$

Every node has exactly d outgoing and d incoming edges, so the graph is d-regular.

Pseudocode: De Bruijn digraph $\mathrm{DB}(d,k)$

```
Input:  d (degree), k (string length), n = d^k
Output: directed d-regular graph on {0, ..., n-1}

for each node b in {0, ..., n-1}:
  prefix = b mod d^{k-1}          # drop the leading digit
  for c = 0, 1, ..., d-1:
    add edge  b -> prefix * d + c  # shift left, append c
```

Routing. To route from s to t, simply append the digits of t one at a time, most-significant first. After exactly k steps, all digits of s have been shifted off and the current node is exactly t. Every pair (s, t) uses exactly $k = \log_d n$ hops—no path is longer or shorter.

Why this is tight. Because every pair takes exactly k hops, $\bar{h} = k$, and the bound from Step 1 gives $\alpha^* \leq 1/k$. To see that $\alpha = 1/k$ is achievable, route all demands simultaneously along their k-hop De Bruijn paths. The shift symmetry of the construction ensures every link carries the same load: total traffic injected is αnd, each unit travels k hops, so total link usage is αndk, spread evenly across nd links. Each link carries αk; setting $\alpha k = 1$ saturates every link simultaneously, so $\alpha^* = 1/k = 1/\log_d n$, matching (3) exactly.

Sanity check. When $d = n - 1$ every node connects directly to every other node (K_n). Then $k = 1$, $\bar{h} = 1$, and $\alpha^* = 1$—all traffic is routed in one hop with no congestion, as expected.

What if n is not a power of d? De Bruijn requires $n = d^k$. For general n, a random d-regular graph achieves an average hop count of approximately $\log_{d-1} n$, giving the same asymptotic guarantee.

Why not a leaf-spine topology? A leaf-spine network achieves diameter 2 but requires the spine switches to have degree n, far exceeding the degree budget of d. If the spine nodes are drawn from the same n-node pool (as required here, since there are no additional switches), each spine can serve at most d hosts, collapsing to the complete graph K_n when $d = n - 1$. For $d \ll n$, the degree constraint makes a true leaf-spine infeasible, and De Bruijn's $\log_d n$ diameter is the best achievable.

Summary. *Use the De Bruijn digraph. It minimizes average hop count to $\log_d n$ via a simple shift-and-append construction, achieving $\alpha^* = 1/\log_d n$, which matches the impossibility lower bound. No d-regular topology can do better.*

Part 2 – Arbitrary (Hose) Traffic (50 pts)

Motivation. Unlike Part 1, the traffic matrix here is not fixed in advance. In practice—for example, in MoE inference where each token is routed to a different expert GPU—the demand pattern is unpredictable and changes every forward pass. We therefore want a topology that performs well for *any* T in the hose polytope, not just a symmetric all-to-all pattern. For a given T, we solve for the best topology and routing jointly.

We fix $n = 8$, $d = 4$, and the hose polytope

$$\mathcal{T}_{\text{hose}} = \left\{ T \geq 0 : T_{ii} = 0,\ \sum_j T_{ij} \leq 4,\ \sum_i T_{ij} \leq 4 \right\}.$$

Given $T \in \mathcal{T}_{\text{hose}}$, we jointly choose topology G and routing to maximize $\alpha^*(G, T)$.

MILP formulation

We solve this as a Mixed-Integer Linear Program (MILP). The decision variables are:

- $x_{ij} \in \{0, 1, \ldots, 4\}$: number of directed links from node i to node j (the topology).
- $f_{ij}^{s,t} \geq 0$: flow of demand pair (s, t) on link (i, j) (the routing).
- $\alpha \geq 0$: the concurrent flow value to maximize.

Objective. max α.

Constraints.

$$x_{ii} = 0 \qquad \text{(no self-loops)} \tag{4}$$

$$\sum_{j \neq i} x_{ij} = 4 \qquad \forall i \quad \text{(out-degree} = 4) \tag{5}$$

$$\sum_{i \neq j} x_{ij} = 4 \qquad \forall j \quad \text{(in-degree} = 4) \tag{6}$$

$$\sum_{(s,t)} f_{ij}^{s,t} \leq x_{ij} \qquad \forall i \neq j \quad \text{(link capacity)} \tag{7}$$

$$\sum_{j} f_{vj}^{s,t} - \sum_{i} f_{iv}^{s,t} = \alpha T_{s,t} \cdot \mathbf{1}[v{=}s] - \alpha T_{s,t} \cdot \mathbf{1}[v{=}t] \qquad \forall (s,t), v \quad \text{(flow conservation)} \tag{8}$$

Since $T_{s,t}$ is a given constant, the products $\alpha \cdot T_{s,t}$ are linear in α, making this a valid LP/MILP.

What each constraint does. Constraints (5)–(6) ensure the chosen x is a valid 4-regular topology. Constraint (7) says total flow on a link cannot exceed the number of parallel links wired there. Constraint (8) is standard flow conservation: for each demand pair (s, t), source s injects $\alpha T_{s,t}$ and sink t absorbs it, while all other nodes pass it through.

The MILP simultaneously decides:

1. **Wiring**: how to allocate each node's 4 outgoing link slots among the other 7 nodes.
2. **Routing**: how to split each demand (s, t) over multi-hop paths given that wiring.

Implementation (Gurobi Python API)

Listing 1: `solution.py`: Gurobi MILP for Part 2 (core solver).

```python
def solve_best_topology(
    T: np.ndarray,
    d: int = 4,
    allow_parallel_links: bool = True,
    time_limit_s: float = 60.0,
    verbose: bool = False,
) -> SolverResult:
    """Solve the topology-and-routing MILP for a given traffic matrix T.

    Decision variables
```

```python
    ------------------
    x[i,j]   : integer in [0, d], # directed links from i -> j (x[i,i] = 0)
    g[t,i,j] : continuous >= 0, aggregated flow towards destination t on (i,j)
    alpha    : continuous >= 0, concurrent-flow scalar (objective)

    Constraints
    -----------
    * x is a d-regular nonnegative integer matrix with zero diagonal
    * Per-edge capacity: sum_t g[t,i,j] <= x[i,j]
    * Flow conservation for each destination t:
        - At every source v != t:   out(g_t, v) - in(g_t, v) = alpha * T[v,t]
        - At destination t:         in(g_t, t) - out(g_t, t) = alpha * sum_v T[v,t]
    """
    T = np.asarray(T, dtype=float)
    n = T.shape[0]
    assert T.shape == (n, n)
    assert np.allclose(np.diag(T), 0.0), "T_ii must be 0"

    nodes = range(n)
    # Active destinations: anyone who receives some traffic
    dests: List[int] = [t for t in nodes if T[:, t].sum() > 0]

    m = gp.Model("best_topology")
    if not verbose:
        m.Params.OutputFlag = 0
    m.Params.TimeLimit = time_limit_s

    # --- topology variables ---
    x_ub = d if allow_parallel_links else 1
    x = m.addVars(nodes, nodes, vtype=GRB.INTEGER, lb=0, ub=x_ub, name="x")

    # --- destination-aggregated flow variables ---
    g = m.addVars(dests, nodes, nodes, lb=0.0, name="g")

    # --- alpha ---
    alpha = m.addVar(lb=0.0, name="alpha")

    # --- no self-loops in topology, and no flow on self-links ---
    for i in nodes:
        m.addConstr(x[i, i] == 0, name=f"no_self_{i}")
    for t in dests:
        for i in nodes:
            m.addConstr(g[t, i, i] == 0, name=f"no_self_g_{t}_{i}")

    # --- d-regularity (in and out) ---
    for i in nodes:
        m.addConstr(
            gp.quicksum(x[i, j] for j in nodes) == d, name=f"outdeg_{i}"
        )
        m.addConstr(
            gp.quicksum(x[j, i] for j in nodes) == d, name=f"indeg_{i}"
        )

    # --- capacity: sum_t g[t,i,j] <= x[i,j] ---
    for i in nodes:
        for j in nodes:
            if i == j:
                continue
```

```python
            m.addConstr(
                gp.quicksum(g[t, i, j] for t in dests) <= x[i, j],
                name=f"cap_{i}_{j}",
            )

    # --- flow conservation per destination ---
    for t in dests:
        total_in = float(T[:, t].sum())  # sum_v T[v,t]
        for v in nodes:
            out_v = gp.quicksum(g[t, v, w] for w in nodes if w != v)
            in_v = gp.quicksum(g[t, u, v] for u in nodes if u != v)
            if v == t:
                # destination absorbs total demand
                m.addConstr(in_v - out_v == alpha * total_in,
                            name=f"cons_{t}_dst")
            else:
                # source v injects its own demand alpha * T[v,t]
                m.addConstr(out_v - in_v == alpha * T[v, t],
                            name=f"cons_{t}_{v}")

    # --- objective ---
    m.setObjective(alpha, GRB.MAXIMIZE)
    m.optimize()

    # --- unpack solution ---
    adj = np.zeros((n, n), dtype=int)
    for i in nodes:
        for j in nodes:
            adj[i, j] = int(round(x[i, j].X))

    flows: Dict[int, np.ndarray] = {}
    for t in dests:
        F = np.zeros((n, n), dtype=float)
        for i in nodes:
            for j in nodes:
                if i == j:
                    continue
                F[i, j] = g[t, i, j].X
        flows[t] = F

    return SolverResult(
        alpha=float(alpha.X),
        adjacency=adj,
        dest_flows=flows,
        model_status=m.Status,
    )
```

Implementation note: destination-aggregated flows. The per-pair formulation above has $O(n^2)$ flow variables per link. To stay within Gurobi's free-license variable limit, we aggregate flows by destination: let $g_{ij}^t = \sum_s f_{ij}^{s,t}$ be the total flow on link (i, j) heading toward destination t. The conservation constraint becomes: every node $v \neq t$ sends out $\alpha T_{v,t}$ more than it receives (it is a source for the t-flow), and t absorbs the rest. This reduces variables by a factor of n and gives the same optimal α.

Results

We tested three traffic matrices:
1. **Cyclic permutation** $\times 4$: node i sends all 4 units to node $(i+1) \bmod 8$.
2. **Uniform hose**: $T_{ij} = 4/7$ for all $i \neq j$.
3. **Mixed**: two heavy demands plus a thin uniform background.

Traffic matrix	α^*	Topology found
Cyclic permutation $\times 4$	1.000	4 parallel links $i \to (i{+}1) \bmod 8$
Uniform hose $T_{ij} = 4/7$	0.700	4-regular digraph, all 32 links utilized
Mixed heavy + background	1.000	3 direct links on heavy pairs, multi-hop background

Checking the uniform-hose result. In any 4-regular graph on 8 nodes, each node has 4 neighbors at distance 1 and the remaining 3 nodes are at distance at least 2. So:

$$\bar{h} \;\geq\; \frac{4 \cdot 1 + 3 \cdot 2}{7} = \frac{10}{7}.$$

Plugging into the hop bound: $\alpha^* \leq 1/\bar{h} \leq 7/10 = 0.7$. The MILP returns exactly 0.7, so the topology it found is provably optimal and every link is fully utilized.

Interpreting the results.

- When T is an integer hose matrix (row/column sums $= 4$), direct routing is optimal: wire $x_{ij} = T_{ij}$ links and route one hop, giving $\alpha = 1$.
- When T is fractional (like $T_{ij} = 4/7$), no integer wiring matches the demand directly. The MILP spreads links across multiple destinations and routes multi-hop, producing a well-connected topology consistent with the De Bruijn intuition from Part 1.

Deliverables

- `report.tex / report.pdf` – this document.
- `solution.py` – Gurobi MILP implementation with three test cases and a constraint verifier.
- `README.md` – build and run instructions.